Live Show Radar · Components map

Every actor, left to right: what starts it, what it is given, what it knows, what it does, where the model is involved, and what comes out. No show is hard-coded — every one is found at runtime from live sources.

live-show-radar-pink.vercel.app

All AI on gemini-3.8-flash via @google/genai - no other model anywhere

Actor / Platform	Trigger	Input	Context	Processing	AI	Output
Schedulers GitHub Actions + Vercel Cron	**Every 30 minutes** Vercel daily cron 08:00 IST as backup		**CRON_SECRET** repo secret + Vercel env	**POST /api/scan** bearer token; fails the step on non-2xx		**Scan run opened** scan_runs row
Scan orchestrator Next.js route on Vercel	**GET or POST /api/scan** bearer, or an admin session	**Artists due now** 15 max, starred first	**300 s function ceiling** bounded batch keeps runs seconds long	**Fan out, then close the run** counts + per-source errors recorded		**Rechecked in 2 h or 6 h** starred / everyone else; JSON summary
Per-artist sources Google News RSS - Tavily - Gemini	**Scan opens**	**Every name and alias** one query each	**when:14d, India feed** Tavily for starred and new artists	**Fetch in parallel** concurrency 5, 10 s timeout, 429 backoff	**Grounded search** optional, starred only	**Items tied to an artist**
Platform sweep Tavily, artist-agnostic	**6 h since the last**	**Four broad queries** new concerts, early bird, festivals	**include_answer: false** District, BookMyShow, Skillbox, +2	**Search without an artist** stored with artist_id null	**Match to the whole list** finds shows nobody searched for	**Items for any artist**
Filter Deduplication	**Items fetched**	**All items this scan**	**url_hash unique key** all previous scans; 30-day limit	**Canonicalise + SHA-256** drop duplicates and stale items		**source_items** only new items continue
Show extractor Gemini Task 1	**Unprocessed items**	**Artist + aliases** up to 25 items per call	**Appendix B JSON schema** today's date; full artist list	**Structured output** Zod validated, one retry, then skip	**Extract candidates** reports near misses honestly	**Show candidates** evidence quote + source URL
Triage + matcher Rules in code	**After extraction**	**Show candidates**	**Keep rules** India, confidence 0.7+, upcoming	**Build dedup_key** artist | normalised city | date		**Accepted** new show, or an existing match
Rejection log What it refused	**A candidate fails**	**Candidate + reason**	**Six reasons** not India, tribute, namesake, past	**Store the evidence** the quote it was judged on		**rejections** shown on /filtered, never alerted
Reconciler Gemini Task 2	**dedup_key already exists**	**Stored show + candidate**	**Null means unreported** never cleared	**Merge stated fields only**	**Same show? changed?** one material change	**Show updated** flagged when worth an alert
Prioritiser + writer Rules in code + Gemini Task 3	**New show or change**	**City, confidence, early bird**	**P0 Bengaluru + early bird** P1 / P2 / P3 by rule	**Priority set in code** the model never sets it	**Write the alert** 600 visible chars, explains priority	**Alert ready** P3 stops here: dashboard only
Early-bird reminder Every scan	**Deadline within 24 h** while the status is still open	**The show + its deadline**	**Deadline is the change_ref** a moved deadline may alert again	**Route to the alert writer**	**Write the reminder** Gemini Task 3	**One reminder per deadline**
Notifier Telegram Bot API - Resend	**Alert written**	**HTML + chat ID or email**	**alert_key unique in Postgres** show | type | channel | change	**Claim the row, then send** the constraint is the lock		**Karan is alerted** alerts row with delivery status
Karan Bengaluru	**Phone buzzes**	**[BUY TODAY] ...**		**Reads and decides**		**Buys the early bird** before the window closes
Artist sync Spotify Web API	**Daily Vercel cron** or Sync Spotify in the admin UI	**GET /me/following** type=artist, limit=50, cursor-paginated	**OAuth, scope user-follow-read** tokens server-side and refreshed; dev mode allows 5 users	**Insert, reactivate, retire** unfollowed artists deactivated, never deleted; manual artists untouched		**artists, source spotify** name and artwork
Artist search and import Spotify catalogue - CSV	**Admin types a name** or pastes a list / imports CSV	**Free text, 2+ characters** or one name per line with aliases	**App-level token** existing list flags duplicates; works with no Spotify at all	**Debounced search** stale responses discarded; conservative name match for artwork		**artists, source manual** scanned exactly like Spotify ones
Artist suggestions Gemini	**Admin presses Suggest**	**Everyone followed** starred weighted first	**Spotify recommendations API** closed to apps created after Nov 2024, so Gemini does this	**Exclude the known** artwork looked up in the catalogue	**Suggest similar artists** likely to play India	**artist_suggestions** Add or Dismiss on /artists
Database Postgres on Neon	**Every stage**	**Reads and writes** from the whole pipeline	**Drizzle ORM, SQL migrations** all timestamps stored in UTC	**Nine tables** users, artists, scan_runs, source_items, shows, alerts, ai_calls, suggestions, rejections		**One source of truth** uniqueness on url_hash, dedup_key, alert_key
Dashboard and logs Next.js pages	**A request from the browser**	**City, priority, month, reason** as query parameters	**Live rows on every request** times shown in Asia/Kolkata	**Seven pages** dashboard, calendar, artists, filtered, alerts, scans, how it works		**The interface** every AI call and refusal visible
Access control Admin password	**Any page or admin action**	**ADMIN_PASSWORD** or a bearer token for the schedulers	**Cookie holds a hash** never the password itself	**Middleware gates every page** /api/scan keeps its own bearer check so a scheduler fails loudly		**A session, or /admin** sign-in page

Deployment — code to production

GitHub
repository + Actions cron → **Vercel**
Next.js 16, 300 s ceiling → **Neon Postgres**
Drizzle migrations → **Live app**
scans every 30 min

= AI component · = System step · = Output · = Human step · = Refused / not alerted · = Context

Priority is decided in code, never by the model. The model judges language; code decides consequences.